

TITLE OF THE INVENTION

SYSTEM AND METHOD FOR HOST-MANAGED MULTI-CLIENT MODEL CAPABILITY ROUTING, SHARED UNIFIED MEMORY STORAGE, AND COORDINATED SPECULATIVE DECODING

CROSS-REFERENCE TO RELATED APPLICATIONS

Not applicable.

STATEMENT REGARDING FEDERALLY SPONSORED RESEARCH OR DEVELOPMENT

Not applicable.

BACKGROUND OF THE INVENTION

1. Field of the Invention The present disclosure relates generally to machine learning runtime architectures and memory management subsystems. More specifically, it relates to systems and methods for dynamically arbitrating machine learning model selection based on client capability intent, managing zero-copy single-instance shared weight storage in unified memory architectures, and coordinating speculative decoding pipelines across heterogeneous client applications.

2. Description of the Related Art In modern computing systems, particularly unified memory architectures wherein central processing units (CPUs), graphics processing units (GPUs), and neural accelerators share a common physical address space, local execution of large machine learning models (e.g., autoregressive transformer models and large language models) has become increasingly prevalent. Conventional deployment paradigms rely on application-managed execution: individual application containers, sandboxed user-space processes, and standalone scripts independently manage model discovery, disk storage, and runtime invocation.

This decentralized paradigm introduces severe technical inefficiencies and hardware contention:

- **Storage Duplication:** Redundant storage allocation occurs when disparate applications download identical model weight tensors into isolated application sandboxes, prematurely exhausting solid-state drive (SSD) storage subsystems.
- **Fragile Formatting Dependencies:** Client applications typically hardcode explicit repository identifiers and static chat templates, causing runtime failures when model architectures evolve or when an underlying model lacks support for specific target device hardware.
- **Memory Contention and Thrashing:** Uncoordinated allocations of unified system memory result in severe memory contention, excessive paging, and out-of-memory kernel panics when multiple client applications invoke local inference pipelines simultaneously.
- **Lack of Multi-Model Coordination:** Advanced inference acceleration techniques, such as speculative decoding requiring synchronized low-parameter draft models and high-parameter target models, cannot be effectively coordinated across disparate client

processes without substantial memory bloat, race conditions, and GPU dispatch queue contention.

Consequently, there is a clear technical need for a centralized host-managed runtime broker capable of intent-based model resolution, zero-copy shared storage utilization, and host-orchestrated speculative decoding on unified memory architectures.

BRIEF SUMMARY OF THE INVENTION

The present invention addresses these technical challenges by providing a centralized host runtime broker configured to arbitrate hardware resource allocation, model weights, and inference pipelines across a plurality of independent client processes.

In one aspect of the invention, the host broker receives an inference request comprising an intent tuple rather than a static model file identifier. The intent tuple specifies operational constraints, such as an execution modality, an upper-bound physical memory budget, and a quality tier. The host broker evaluates this intent tuple against a registry of locally stored model weights and dynamic hardware telemetry (e.g., currently committed unified memory, available purgeable cache, and accelerator dispatch queue depth) to deterministically select an optimal model configuration.

In another aspect of the invention, model weight tensors are maintained in a centralized, configurable host storage path. The host broker maps these weights into a shared unified memory space using virtual memory primitives (e.g., POSIX mmap), allowing multiple client applications running in isolated sandboxes to execute inference against shared physical tensor pages without duplicating weights in disk storage or system RAM.

In another aspect of the invention, the host broker orchestrates a multi-model speculative decoding pipeline. The host broker couples a high-parameter target model with an aligned low-parameter draft model fitting within the resolved memory budget. The draft model generates a speculative sequence of tokens at high velocity, followed by batched parallel verification by the target model executed within a unified hardware acceleration framework, reducing inference latency while maintaining strict memory guarantees.

BRIEF DESCRIPTION OF THE SEVERAL VIEWS OF THE DRAWINGS

- **FIG. 1** is a block diagram illustrating the overall system architecture, including client applications, a centralized host model broker runtime, single-instance shared storage, and a unified memory execution subsystem.
- **FIG. 2** is a flowchart illustrating the operational logic for resolving an intent-based capability tuple against dynamic hardware telemetry constraints.
- **FIG. 3** is an architectural diagram illustrating zero-copy single-instance shared memory mapping across multiple isolated client application sandboxes.
- **FIG. 4** is a message sequence diagram illustrating the host-orchestrated speculative decoding execution loop between a client process, a host broker, a draft model, and a target model.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

Referring to the drawings in detail, exemplary embodiments of the present invention provide a system and method for dynamic model capability routing, single-instance storage management, and host-orchestrated speculative decoding.

FIG. 1 illustrates an exemplary computing environment 100 configured in accordance with the present disclosure. Computing environment 100 comprises client applications, such as a containerized sandboxed application 102a and an independent script or command-line client 102b. Each client application communicates with a centralized host model broker runtime 106 via an inference request interface 104. In an exemplary embodiment, inference request interface 104 exposes an OpenAI-compatible HTTP REST protocol over a local loopback network interface (e.g., localhost) or an inter-process communication (IPC) domain socket.

Rather than specifying a static model repository tag or concrete file path, client applications 102a, 102b supply an intent tuple within the inference request payload. The intent tuple specifies multidimensional operational constraints, such as execution modality (e.g., chat, code, vision), a maximum memory allocation threshold (e.g., ram_gb_max), and a quality tier (e.g., compact, balanced, max).

Host model broker runtime 106 includes a capability resolution engine 108, a hardware telemetry monitor 110, a host-owned template and tokenizer normalization engine 112, a speculative decoding engine 114, and a unified memory execution arbiter 116. Hardware telemetry monitor 110 continuously polls host kernel interfaces to extract real-time system metrics, including committed unified memory, purgeable memory pressure, and hardware accelerator dispatch queue occupancy.

Host-owned template normalization engine 112 maintains model-specific formatting specifications, chat templates, and tokenizers. When an inference request is accepted, template normalization engine 112 transforms the client payload into the exact token syntax required by the resolved model weights, insulating client applications 102a, 102b from variations in model prompt formatting.

Single-instance shared storage 118 represents a centralized host storage path storing model weights, such as draft weight tensors 122a and target weight tensors 122b. Shared storage 118 avoids duplicating tensor files across individual client application sandboxes. Unified memory execution arbiter 116 maps weights from shared storage 118 into shared unified memory 120, making tensor pages available in page cache 124a and shared context 124b for direct execution on unified compute hardware 126 (e.g., CPU, Metal GPU, and Neural Engine cores).

FIG. 2 depicts the operational process flow 200 executed by capability resolution engine 108. At step 202, host model broker runtime 106 receives an inbound inference request over interface 104 containing an intent tuple specifying modality, memory budget, and quality tier.

At step 204, hardware telemetry engine 110 queries host kernel virtual memory subsystems (such as Darwin sysctl or Mach VM interfaces) to evaluate current physical memory headroom,

paging activity, and hardware accelerator load. At step 206, the broker queries a local model manifest to filter available candidate models matching the requested modality constraint.

At step 208, the engine evaluates whether a speculative decoding candidate pair is supported for the model family. If affirmative, step 210 calculates the composite footprint of both draft model M_d and target model M_t . If speculative decoding is not available, step 212 identifies a standalone target model footprint.

At step 214, the broker evaluates whether the calculated composite footprint satisfies the constraint:

$$\text{Footprint} \leq \min(\text{ram_gb_max}, \text{Available_Unified_DRAM})$$

If the constraint is satisfied, the configuration is approved. If the memory constraint is violated, step 216 triggers an automated fallback policy: stepping down quantization levels (e.g., from 8-bit to 4-bit precision), disabling speculative decoding to run the target model standalone, or evicting least-recently-used (LRU) model weights from memory. At step 218, the broker outputs an execution plan specifying model weights, quantization schemes, and memory allocations.

FIG. 3 illustrates the zero-copy shared storage and memory mapping architecture 300. Multiple client applications 102a, 102b, and 102c execute within independent process sandboxes, isolated from direct access to underlying model storage directories.

Model weight tensors are stored as single physical files within configurable host storage 118. Upon execution initialization, host broker 106 issues system-level virtual memory mapping calls 302 (e.g., POSIX `mmap()` with `MAP_SHARED` flags). The host kernel maps underlying non-volatile disk blocks into read-only physical memory pages 304 within shared unified memory space 120.

Because the system employs a unified memory architecture, CPU cores 126a, GPU cores, and Neural Engine cores 126b reference the same physical DRAM pool. Host broker 106 passes zero-copy buffer pointers 306 directly to hardware acceleration frameworks (e.g., Apple Silicon MLX and Metal APIs). Multiple concurrent client processes read the identical physical tensor pages without copying weights across IPC boundaries, preventing container disk bloat and keeping RAM utilization strictly bound to active working sets.

FIG. 4 is a message sequence diagram illustrating host-orchestrated speculative decoding 400. Client process 102 submits an inference generation request over interface 104. At step 402, template normalization engine 112 formats the prompt using model-specific chat templates and tokenizes the input text into token IDs.

At step 404, host broker 106 dispatches input token IDs to draft model 122a. At step 406, draft model 122a executes an autoregressive generation loop on GPU 126b. Because draft model 122a has a smaller parameter footprint (e.g., 0.5 billion parameters compared to 1.5 billion

parameters), token generation proceeds at high velocity. Draft model 122a produces K candidate tokens.

At step 408, the K candidate tokens are returned to broker 106 within shared unified memory 120. At step 410, broker 106 constructs a parallel evaluation batch comprising the context tokens and the K candidate tokens, dispatching the batch to primary target model 122b.

At step 412, target model 122b evaluates the candidate tokens in a single parallel forward pass on GPU 126b. At step 414, target model 122b validates N candidate tokens (where $N \leq K$) and appends a corrective token based on its output probability distribution.

At step 416, broker 106 commits the verified token sequence to shared key-value (KV) cache 124b. At step 418, broker 106 streams the accepted tokens back to client process 102 via a Server-Sent Events (SSE) stream, while recycling context into draft model 122a for the subsequent generation cycle.

CLAIMS

WE CLAIM:

1. A computer-implemented method for arbitrating machine learning model execution on a computing device having a unified memory architecture, the method comprising:
 - receiving, by a host runtime broker from an independent client process over a local inter-process interface, an inference request comprising an intent tuple defining at least an operational modality, a maximum memory budget, and a quality tier, wherein the inference request is devoid of a static model file identifier;
 - querying, by the host runtime broker, hardware telemetry registers of the computing device to determine currently available physical unified memory shared across a central processing unit (CPU) and a graphics processing unit (GPU);
 - dynamically resolving, by the host runtime broker, the intent tuple against a local manifest of candidate model weight tensors to identify an executable model configuration that satisfies the operational modality and has an aggregate memory allocation footprint within both the maximum memory budget and the currently available physical unified memory;
 - mapping, from a centralized shared storage directory, weight tensors corresponding to the resolved executable model configuration into a shared unified memory space accessible by the GPU via a zero-copy memory-mapped file interface; and
 - executing inference on the GPU using the memory-mapped weight tensors to fulfill the inference request for the independent client process.
2. The method of claim 1, wherein dynamically resolving the intent tuple further comprises:
 - identifying a multi-model speculative decoding pair comprising a draft model and a target model belonging to a common architectural family; and

- allocating execution of both the draft model and the target model within the shared unified memory space, wherein the aggregate memory allocation footprint comprises the combined weights of the draft model and the target model.
- 3. The method of claim 2, further comprising:
 - generating, via the draft model executing on the GPU, a candidate sequence of speculative tokens;
 - verifying, via the target model executing on the GPU, the candidate sequence of speculative tokens in a single parallel verification step; and
 - transmitting an accepted subset of the candidate sequence of speculative tokens to the independent client process over the local inter-process interface.
- 4. The method of claim 1, wherein the centralized shared storage directory is accessible to a plurality of distinct client application sandboxes, and wherein the weight tensors are stored as a single physical disk instance without duplication across the distinct client application sandboxes.
- 5. The method of claim 1, further comprising:
 - maintaining, within the host runtime broker, a plurality of model-specific chat templates and tokenizers; and
 - formatting, by the host runtime broker, a prompt payload received in the inference request using a normalized template corresponding to the resolved executable model configuration prior to executing inference, wherein the client process is decoupled from template specifications of the resolved model configuration.
- 6. The method of claim 1, wherein the local inter-process interface comprises an HTTP server listening on a local loopback address providing an OpenAI-compatible application programming interface (API) endpoint.
- 7. The method of claim 1, wherein mapping the weight tensors into the shared unified memory space comprises initializing read-only memory pages shared across concurrent inference requests originating from different sandboxed client applications.
- 8. A system for arbitrating machine learning inference operations, comprising:
 - a unified memory architecture comprising a physical dynamic random-access memory (DRAM) pool shared across at least a central processing unit (CPU) and a hardware accelerator;
 - a non-volatile storage device storing a plurality of model weight tensors in a centralized shared directory; and
 - a processor executing a host model broker configured to:
 - expose an inference endpoint to a plurality of client processes;
 - receive from a first client process an inference request specifying a desired capability constraint set including a maximum RAM limit and an execution modality;
 - measure dynamic allocation state of the physical DRAM pool;
 - select, from the centralized shared directory, an optimal model tensor set satisfying the capability constraint set and capable of resident execution within the dynamic allocation state;

- map the optimal model tensor set directly into the physical DRAM pool using a zero-copy virtual memory interface accessible by the hardware accelerator; and
 - dispatch inference generation tasks to the hardware accelerator on behalf of the first client process.
- 9. The system of claim 8, wherein the host model broker is further configured to:
 - select a speculative decoding pair comprising a primary target model and an auxiliary draft model having a parameter count smaller than the primary target model; and
 - orchestrate iterative speculative token generation by the auxiliary draft model and batched token verification by the primary target model within the physical DRAM pool.
- 10. The system of claim 8, wherein the non-volatile storage device maintains only a single instance of the optimal model tensor set, and wherein multiple client processes execute inference against the single instance without duplicating the optimal model tensor set into isolated client storage containers.
- 11. A non-transitory computer-readable storage medium comprising instructions that, when executed by a processor of a computing device having a unified memory architecture, cause the processor to perform operations comprising:
 - receiving an inference request from an application process over an inter-process communication channel, the request defining capability intent parameters comprising at least a modality and a client-specified memory constraint;
 - inspecting hardware registers to determine available memory in a unified memory subsystem shared by a CPU and an accelerator;
 - resolving the capability intent parameters to a concrete model weight representation stored in a single-instance shared repository, wherein the concrete model weight representation requires memory within the client-specified memory constraint and the available memory;
 - exposing the concrete model weight representation to the accelerator via a shared memory-mapped buffer; and
 - generating inference output tokens via the accelerator for return to the application process.
- 12. The non-transitory computer-readable storage medium of claim 11, wherein the concrete model weight representation comprises a curated draft model and target model pair configured for speculative decoding execution within an MLX framework.

ABSTRACT OF THE DISCLOSURE

A system, method, and non-transitory computer-readable medium are disclosed for host-managed model capability routing, single-instance unified memory storage, and coordinated speculative decoding. A centralized host runtime broker receives an inference request containing an intent tuple specifying abstract requirements such as modality, RAM budget, and quality tier, rather than hardcoded model identifiers. The host broker monitors dynamic hardware telemetry and unified memory pressure to resolve the intent tuple to optimal model weights stored in a centralized shared directory. Model weights are mapped into shared

unified memory using zero-copy memory-mapped file primitives, preventing redundant containerized disk and RAM duplications. Furthermore, the host broker automatically pairs draft and target models for unified speculative decoding execution, decoupling client applications from template formatting, weight management, and hardware contention.